Injective Audit Report

Version 1.0

November 28, 2024

Injective Audit Report

Prepared by:
Andrei Stefan
Matei Graura
Valentin Craciun

Contents

- About Us
- Disclaimer
- Risk Classification
  - Impact
  - Likelihood
- Executive Summary
  - About Injective
  - Scope
- Findings Summary
- Findings

About Us

The team of Independent Security Researchers formed by Matei Graura, Stefan Andrei and Valentin Craciun has extensive experience in web3 technologies as they have been part of a Layer 1 development team, and they have been part of a tier 1 security team where they had the roles of CTO and Lead auditors. In their career so far they have added production code in node infrastructure, web assembly VMs and audited different DeFi protocols. These include lending protocols, AMMs, NFT standards and marketplaces, derivative exchanges. The team is well-equipped to provide robust security solutions and insights within the web3 space, thanks to their extensive hands-on experience.

Protocol Summary

The Injective Peggy bridge consists of three main components:

- Peggy Ethereum Contract
- Peggo Orchestrator module
- Peggy Injective Module

The **Peggy Ethereum Contract** and the **Peggy Injective Module** were in scope of this report.

Peggy Contract

The Peggy contract enables seamless two-way transfers of ERC-20 tokens between Ethereum and the Injective Chain. The Injective Peggy bridge stands apart from traditional bridge solutions as it operates as a decentralized, trustless bridge managed exclusively by Injective validators. The bridge's security is anchored in Injective Chain's proof of stake mechanism, where token deposits and withdrawals are executed only after receiving attestations from validators representing a minimum two-thirds of the consensus staking power.

Peggy Module

The Peggy module's core function is to create new tokens on Injective Chain when ERC-20 tokens are deposited from Ethereum, and burn tokens when they are withdrawn back to Ethereum. To promote validator integrity and efficiency, the module incorporates various economic incentives including penalty mechanisms, token-based rewards, and fees charged on withdrawals.

Disclaimer

The team has made all efforts to find as many vulnerabilities in the codebase within the given scope and timeframe. However it holds no responsabilities for the findings presented in this document. The review and report should be treated on an “as-is” basis, without any implied warranties or assurances for the completeness of the security posture of the project. This report does not endorse any specific project or team, nor does it guarantee the complete security of the project.

Risk Classification

		Impact		
		High	**Medium**	**Low**
	High	High	High	Medium
Likelihood	**Medium**	High	Medium	Low
	Low	Medium	Low	Low

Impact

- **High** - significant loss of assets, funds directly at risk, severe disruption of functionality
- **Medium** - moderate loss of assets, funds indirectly at risk, moderate disruption of functionality
- **Low** - funds are not at risk

Likelihood

- **High** - direct attack vector; easy to perform or the cost is low relative to funds at risk
- **Medium** - only conditionally possible or incentivized attack vector, but still relatively likely
- **Low** - requires too many or too unlikely assumptions; little-to-no incentive

Executive Summary

About project

Injective is the blockchain built for finance.

Injective is a high-performance, interoperable layer-one blockchain for building premier Web3 financial applications.

Injective uniquely provides developers with modular, adaptable components to build sophisticated applications beyond what other blockchains can support. Its optimized architecture and advanced cross-chain connectivity create a powerful network infrastructure, designed to facilitate the secure and efficient transition of global finance to blockchain technology.

Scope

Project	Injective - Peggy
Repository	Peggy repository
Commit	b094a25889a1270872b3584f2a232cce7a2daccc
Methodology	Manual review

Project	Injective - Contract
Repository	Peggo repository
Commit	9e702f1ec4539746217c5352584d64d7038c7e83
Methodology	Manual review

Findings Summary

ID	Description	Severity
1	Economic attack over validator	High
2	SubmitBadSignature logic vulnerable to replay	Medium
3	MEV actor can steal rewards from validator	Medium
4	Malicious gas tokens or hook tokens like erc777 can DOS through out of gas	Medium
5	Decimals wrapped on Injective side can be exploited in case of ERC20Upgradeable	Medium
6	Bridge business logic not taking into account the case of rewards/rebase tokens	Medium
7	Token representation on other chains could create problems for bridge expansion	Medium
8	ECDSA signature is malleable	Low
9	No upper limit for transactions	Low
10	EIP-712 is not fully implemented	Low
11	Validators set out of gas limit could result in blocked flows	Low
12	Gas limits on fixed size batches could result in unprocessed batches	Low
13	Anybody can lock out the owner of Peggy contract	Informational

Findings

1. Economic attack over validators

Severity: High

Status: Acknowledged

In the current flow of events there are multiple smaller issues that, if they are chained, can result in an economic attack over the validators.

- There is no lower limit on the transfer amount that is enforced in the module/smart-contract logic, as long as the transferred amount is bigger than 0, the transfer request will be valid (limit is only enforced in app frontend, which is not enough).
- There is no minimum required fee, as long as the fee is bigger than 0 and not nil is a valid fee amount.
- At the moment because of the logic inside Peggo, every time when a new tx is added using `AddToOutgoingPool`, the new Peggo will request a batch, however the batch will always be formed from just 1 tx. This fact is not aligned with the purpose of batch submitting which should process/digest multiple transactions at the same time.

All of these issues could be combined by a user that will send multiple transactions of very low value on the bridge, a small amount of a few hundreds USDT combined with a few INJ (gas prices are very low on Injective compared to ETH) could result in tens of thousands of txs that a validator will execute, as each transfer of eth cost approximately 30usd at this time it could completely drain the funds of the orchestrator that will execute them. The orchestrator will have gained almost nothing for executing those transactions. We are aware that the orchestrators are setting a minimum USD threshold themselves; however, that is a local setting and they should be protected by the business logic of the protocol.

We queried the chain using Dune analytics and have examples of such transactions. Example → https://etherscan.io/tx/0x587f217db364c85e53920bd6b763eefbfb841ca96dedbdd3b14cf73ee360e1ad

Recommendation: Add lower limit for transfer amounts, lower limit for fees and different types of transfer, to be sure you have a distinction between batch transfer and single transfer.

Team Response: Acknowledged, we are improving the economic robustness.

2. SubmitBadSignature logic vulnerable to replay

Severity: Medium

Status: Acknowledged

In the Peggy module, the `SubmitBadSignature` flow where a malicious validator can be punished for trying to submit a malicious signature can be abused by other actors to keep a validator in jail, basically after the validator is getting out of jail (unjail) the previous bad signature can be submitted again to put him in jail mode with the same bad signature. In the normal flow a validator should be punished only once for one bad signature.

Recommendation: Save in storage the malicious signature at the end of the flow to be sure it can not be used multiple times.

PoC -> https://hackmd.io/@QyPimM2nQzSOYG0jHv2aIg/SJQMWfljp

Team Response: Interesting, yes. Not repeatedly punishing malicious validators seems like the correct thing to do. Recommend recategorizing the security level to Medium/Low since this only affects malicious validators.

3. MEV actor can steal rewards from validator

Severity: Medium

Status: Acknowledged

On the solidity side of things, the `submitBatch` function is used to submit a batch of transactions. That function can be called by anyone and that represents a problem as the functions can result in monetary gains for the `msg.sender` as the validators should be economically incentivised to hold the bridge infrastructure. At this moment a batch of 100 transactions (the batch size limit) consumes approximately ~3.25 million gas units, with the current eth prices it means an approximate cost of 220$, if the fees per batch are high enough (calculated profit possibility in a full batch of 5-7X, depends from token to token). The mev infrastructure holder can call the function by observing the tx pool on the eth side and just copy the data field in the tx stealing all the profits a validator could make.

Recommendation: Add a sanity check to ensure only an address that is part of the validators set can call the submitBatch function.

PoC -> https://hackmd.io/@QyPimM2nQzSOYG0jHv2aIg/H1MRVGljT

Team Response: It is actually by design that anyone can relay batches. But yes, as a consequence of that, there exists MEV.

4. Malicious gas tokens or hook tokens like erc777 can DOS through out of gas

Severity: Medium

Status: Not applicable

On the Ethereum side of things, a malicious user could leverage the hooks of an erc777 token or create himself a malicious erc20 token that is just following the erc20 interface that will have an operation to on purpose run out of gas or have a switch that will be turned on through a MEV technique to make functions run out of gas. The purpose of this will be to block batches through spamming multiple txs that will run on purpose in an out of gas scenario and make the batches timeout and basically no confirmations of execution will be received on Injective side.

Recommendation: The easiest solution would be to only allow whitelisted tokens

Team comment: Relayers can decide what batches they want to submit or not. Batches are by token, so one malicious token batch will not block other normal batches

5. Decimals wrapped on Injective side can be exploited in case of ERC20Upgradeable

Severity: Medium

Status: Not applicable

The deployment of the denom on the Injective side is taking into account the decimals of the original erc20 token from ethereum side and saving them in storage, this could represent a problem for ERC20 Upgradeable tokens (like USDC), that could theoretically at any time upgrade their code, if one of the tokens will change their decimals from 18 to say 6, the first user who will notice this and withdraw his denom on ethereum could use this to steal all the liquidity locked in the bridge.

Recommendation: Add a check on the orchestrator, before sending the tx, that metadata is the same between the 2 sides of the bridge.

Team comment: Metadata isn't used for the denom representation in the bank module.

6. Bridge business logic not taking into account the case of rewards/rebase tokens

Severity: Medium

Status: Acknowledged

In Peggy bridge, anybody can deploy any counter denom on Injective. This is dangerous as there are multiple types of erc20 tokens, one type that can expose problems is the case of tokens that accumulate rewards or are having an accumulation/rebase mechanism, let's imagine the following scenario: - Bob is depositing into bridge 10 RT (rewards tokens). - Bob is receiving on Injective chain 10 peggyRT tokens. - In time those 10 tokens are accumulating rewards and are transforming in 11 RT tokens - Bob only has 10 peggyRT tokens on Injective and he is transferring all of them back on Ethereum. - The bridge is transferring back those 10 RT tokens on Bob's address however there is 1 RT token that is blocked in the bridge and it can not be used either by the bridge or by Bob.

The same example can be applied for rebase tokens where the balance of user could be dynamic as the tokens is rebasing and when the Bob is transferring back his 10 peggyRT he can withdraw either all the liquidity from that token or a small fraction comparative with the underlying value those tokens hold before rebasing.

Recommendation: The easiest solution would be to only allow whitelisted tokens

Team comment: Peggy does not support rebasing tokens.

7. Token representation on other chains could create problems for bridge expansion

Severity: Medium

Status: Not applicable

At this moment the Peggy bridge is only deployed on ethereum, however if it will be expanded into other EVM compatibles chains it could possess some risk with the way it's working. The bridge is making some assumptions for contract addresses and decimals, those are mostly valid in the only one chain scenario however in the scenario we have today where there are a lot of chain compatible with EVM that is not true anymore, there could be tokens that possess exactly the same contract address on 2 chains (like Ethereum and Polygon) but have different decimals on those 2 chains, so a user could deposit on 1 chain and withdraw on another one a considerable bigger amount of tokens.

Recommendation: When mapping the tokens also take into consideration to map them based on chain ids, not just on token addresses. Also when creating the name for the denom, right now you are using the prefix peggy and after that adding the contract addresses, be sure you are adding into that formula also the chainId to be sure you are having different tokens for different EVM chains.

Team comment: We don't have plans to integrate Peggy with other EVM chains.

8. ECDSA signature is malleable

Severity: Low

Status: Acknowledged

The way of recovering the public key from ecdsa is not taking into account the ecdsa malleability (check this). It would be best to implement OZ's ECDSA library to solve this bug even if there is not a proper way of weaponizing it.

Recommendation: Use Open Zeppelin ECDSA library instead of ecrecover.

9. No upper limit for transactions

Severity: Low

Status: Acknowledged

An upper limit for transactions, for example no more the 50k tokens for each transfer, is not present. This would represent a protection factor in case of s security incident because it will stop the attacker from stealing all the liquidity in one transaction, giving the team time to react and pause the bridge, otherwise pause functionality is almost rendered useless because the team will be able to trigger it after all the liquidity has been stolen.

Recommendation: Add an upper limit for transfer transactions for the both sides of the bridge

Team comment: Rate limits will be introduced.

10. EIP-712 is not fully implemented

Severity: Low

Status: Acknowledged

On the Ethereum side of the bridge, the EIP-712 is not properly implemented. It is just concatenating the ethereum signing message with the encoded bytes of the signed message. To be fully compliant with EIP-712 it should also take into considerations other factors like chain id, nonce, verifier contract address and so on. The easiest way to implement this properly will be to use OZ implementation from the contracts library.

Recommendation: Implement EIP-712 using Open Zeppelin contracts library.

11. Validators set out of gas limit could result in blocked flows

Severity: Low

Status: Acknowledged

The validators set from ethereum and injective should always be in sync, with the current operations, from the tests that we conducted the limit of validators that the function `updateValset` can operate over is ~128-130, as the numbers of validators on Injective can rise in the future this needs to be taken into account as it can present serious problems like blocking the bridge if the valset will not be updatable in the future because of gas limits from ethereum side.

Recommendation: Take into consideration to have different validators between bridge and injective chain.

12. Gas limits on fixed size batches could result in unprocessed batches

Severity: Low

Status: Acknowledged

Right now the bath size limit is 100, (there are other problems in the code that results in the bridge not operating properly with this batches logic, but to understand this issue we need to ignore those). As every erc20 token can be different, a fix batch size of 100 could result in out of gas txs for `submitBatch` , which will result in blocked batches. A better way of handling this things would be for orchestrators to simulate the txs and request a dynamic number of transfers per batch.

Recommendation: Take into consideration adding dynamic batch sizing.

13. Anybody can lock out the owner of Peggy contract

Severity: Informational

Status: Acknowledged

The contract `Peggy.sol` implements the ownable pattern from `OwnableUpgradeableWithExpiry` . This contract allows for anybody to call the method `renounceOwnershipAfterExpiry` after a certain period of time has passed. This period is evaluated inside the view `isOwnershipExpired`. Given the contract has been deployed for some time, this view returns true, meaning anybody can lock out the owner of Peggy right now. Even if the desired result for this pattern is increased decentralization, it would be advised to be in control of the functions `pause` and `unpause` functions in case of an attack. A malicious user has the possibility to bring the contract's only line of defense down before engaging in malicious actions.

Recommendation: Upgrade the contract and disable the `renounceOwnership` functions by overriding them, especially the one that's externally exposed (`renounceOwnershipAfterExpiry`).

Team comment: Desired behavior.